

计算机学院 高等程序设计 课程实验报告

实验题目：动态数组，深拷贝浅拷贝，string

日期：2026. 4. 16

班级：

Email：18601013891@163.com

实验目的：

1. 掌握动态数组的封装，vector 的使用
2. 掌握浅拷贝和深拷贝的不同
3. 掌握 string，getline（）的使用

实验软件和硬件环境： 笔记本电脑，使用 Clion 进行代码的编辑

实验步骤与内容：

一：

1. (1) Point 不是数组的名字 Point 是类名，代表要创建的对象类型，这里是创建 Point 的动态对象
(2) 不是数组名，points 是只想动态数组的首元素的指针变量，不是数组名，但是这是数组的首
(3) 返回指针就是可直接修改原数组的对应（引用是原对象的别名，支持左值赋值），同时可以避免带来额外的性能开销
(4) (i) 首先没有深拷贝实现：默认浅拷贝会导致多个对象同时共享一块动态内存，析构的时候
(ii) 没有析构函数，释放内存，new[] 分配的动态数组没有使用 delete[] 释放，造成内存泄漏
(iii) 没有下标越界检查，index 超出数组范围会访问非法内存，导致程序崩溃
(iii) 没有使用 const 重载，const 对象无法调用 element 函数，const 的语义不完整

2.

(1)

核心区别：(i) 手动动态数组需要手动创建和回收，容易出错，造成内存管理失败，vector 则是自动
(ii) 手动动态数组大小是固定的，需要手动重新分配，但是 vector 数组就是自动扩容和缩容
(iii) 安全性：手动动态数组没有越界检查，但是 vector 数组就有 at（）函数来抛出异常保护
(iiii) 手动动态数组就是默认的浅拷贝，需要手动深拷贝，但是 vector 数组就是默认的深拷贝，
(iiiii) 手动动态数组就是功能接口 API 只有下表访问，没有内置操作，但是 vector 就有 push_back
(2) vector 对象不表明数组的首地址，普通数组名是编译期常量指针，直接代表数组的首元素地址

二：

(1) 浅拷贝是默认拷贝构造，赋值运算符的行为：只是拷贝对应指针的地址值，不拷贝指针指向的
非法访问

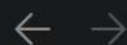
(2) 但是另一个程序就没有对应的浅拷贝问题，其中对应的类成员没有动态分配内存，都是普通值

三：

(1) string，getline 函数的完整解答



文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) ...



practice.cpp ×



practice.cpp > main



```
3 using namespace std;
4 int main()
5 {
6     for (int i = 0; i < 2; i++)
7     {
8         string city, state;
9         getline(&Istr: cin, &Str: cit
10        getline(&Istr: cin, &Str: sta
11        cout << "City: " << city <<
12    }
13    return 0;
14 }
```

问题

输出

调试控制台

终端

端口

State:

(2) 这个语句并非直接赋值给 Employee 对象，而是赋值给了 employee 的指针，语法不允许指针直接赋值给对象这里是指针数组和对象数组的差异

(3) vector, string 与普通生数组的核心不同：

1、内存管理方式中普通数组的站数组大小固定，堆数组需要手动释放，容易内存泄漏，但是 vector, string 是自动管理内存的 (RAII 机制)，构造分配，析构自动释放，不需要手动操作

2. 大小灵活性

普通数组：

编译期确定大小，运行期无法改变

Vector, string 是动态扩容和缩容，赋值的时候自动调整大小，支持 push_back, append 等等动态操作

3. 拷贝和赋值语义，其中普通数组是常量指针，赋值和拷贝都是浅拷贝，就是地址拷贝，多对象共享内存，但是 vector, string 都是默认深拷贝，拷贝赋值后对象独立，修改互不影响

4. 功能和安全上，普通数组没有越界检查，没有内置操作，没有比较，拼接运算符

Vector/string 上支持迭代器和数组内置的丰富操作

(4) vector 和 stringRAII 动态资源管理，这两个都是类的封装，构造的时候分配内存，析构函数自动释放内存，离开作用域自动回收，不会遗漏释放而且赋值扩容的时候自动释放旧内存，重新赋值或者扩容的时候，分配到足够大的新的内存，而且将旧数据深拷贝到新内存，自动释放旧内存，全程没有内存残留，而且深拷贝保证对应的内存独立，赋值的时候拷贝数据内存并且地址，新旧内存完全独立，释放的时候不会重复释放，不会泄漏，同时不需要手动内存释放，从根源避免内存泄

四：个人银行账户管理系统程序



BankAccount ▾

版本控制 ▾



CMakeLists.txt



Account.h



Account.cpp



ma



1 ▶

2

3

4

5

6

7

8

9

10 ▶

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

// TIP 要Run代码, 请按 `<shortcut actionId="Run"/>` 或点击装订

```
#include <iostream>
```

```
#include <vector>
```

```
#include "Date.h"
```

```
#include "Account.h"
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "==== 个人银行账户管理程序 =====<\/pre>" << endl;
```

```
    Date date( y: 2026, m: 1, d: 1);
```

```
    vector<Account> accounts;
```

```
    // 创建账户
```

```
    accounts.emplace_back(date, id: "1001");
```

```
    accounts.emplace_back(date, id: "1002");
```

```
    cout << "已创建账户: 1001、1002" << endl << endl;
```

```
    // 业务操作
```

```
    date = Date( y: 2026, m: 1, d: 5);
```

```
    accounts[0].deposit(date, amount: 10000.0, desc: "工资");
```

```
    accounts[1].deposit(date, amount: 5000.0, desc: "压岁钱");
```

```
    date = Date( y: 2026, m: 2, d: 1);
```

运行



BankAccount



其中日期类封装年月日，实现日期显示比较，利息计算的功能，账户类使用 string 存储银行账号，

实现深拷贝：类中包含指向堆内存的指针成员（new/malloc 分配）

类中包含动态数组的成员

需要拷贝对象，并且要求拷贝后对象与原来对象完全独立

对象会被赋值，函数传参，返回值（触发默认拷贝），并且持有动态资源

核心总结：类中存在动态分配的内存资源，就必须实现深拷贝

结论分析与体会：

1. string 是 c++ 对于数组的安全高级封装，可以动态调整长度，支持深拷贝与丰富运算符和成员函数，彻底解决传统的 char 数组越界，手动拷贝，长度固定都能等缺陷，getline（能够读取包含空格的整行字符串，默认以换行符分割，自定义分隔符，弥补 cin 没有办法读取带空格字符串的不足）
2. emp[0] = new Employee() 合法的前提是 emp 是指针数组，emp[0] 是指针类型，和 new 返回的指针类型匹配，如果是对象数组，指针没有办法直接赋值给对象，语法不成立
3. vector/string 和普通数组的差异，普通数组需要大小固定，手动管理堆内存，默认浅拷贝，没有安全检查，，vector 和 string 都是由类进行封装，支持自动内存管理，动态扩容，默认深拷贝，越界保护，使用更加安全便捷
4. 动态扩容没有内存泄漏的结论，vector、string 都是基于 RAI 的机制，构造分配内存，析构自动释放，，赋值扩容的时候都是先分配新的内存，深拷贝数据，然后释放就得内存，全程没有内存残留，从根源避免泄漏

就实验过程中遇到的问题及解决处理方法，自拟 1—3 道问答题：

1. Getline(cin, str) 和 cin >> str 在读取字符串的时候有什么核心区别？，分别适合那些场景？
Cin >> str 以空格，制表符，换行符作为分隔符，没有办法读取带空格的字符串，而
getline (cin, str) 读取整行内容，保留空格，默认以换行符结束，cin 适合读取没有空格的单词，数字，账号，getline（适合读取姓名，地址，句子等等带空格的整行文本）
2. new[] 和 vector 在拷贝和赋值的时候本质差异就是普通动态数组都是浅拷贝，地址拷贝，多个指针同时指向同一块内存，容易出现重复释放，数据篡改
Vector 赋值和拷贝都是深拷贝，会完整复制数据内容，多个对象内存独立，修改互不影响，更加安全
3. vector 和 string 重新进行赋值和动态改变大小的时候，会不会造成内存泄漏？
二者都是基于 RAI 机制，构造的时候分配内存，析构函数会自动释放内存
扩容和赋值都是先分配新的内存，深拷贝数据，然后自动释放旧内存，没有内存残留
内部实现深拷贝，数据独立，不会出现重复释放或者泄漏